Contents

Keep Me Ready — AI Context Book v1

Prepared: June 2026
Version: 1.0
Folder: `docs/engineering/`
Audience: AI coding assistants working in this repository

> This document is written specifically for AI. It front-loads everything an AI needs to work in this codebase correctly, without re-deriving it from first principles. Read this before opening any source file.

Table of Contents

---

1. Project Snapshot

What it is: Keep Me Ready is a daily bookkeeping practice platform. Learners complete a 10-question drill on a rotating topic, receive per-answer coaching explanations, optionally write a Learning Journal entry, and track progress over time on a dashboard.

Stack: Next.js 16.2.9 · React 19 · TypeScript 5 · Tailwind CSS v4 · Supabase (Auth + PostgreSQL) · Vercel

Live URL: `https://keep-me-ready.vercel.app`
Repository: `git@github.com:tshatiqua-bit/keep-me-ready.git`

Three-sentence architecture summary:
All pages use the Next.js App Router. Anonymous users have their progress stored in `localStorage`; signed-in users have it stored in Supabase. The drill engine is a single client-side state machine (`DrillClient.tsx`) that cycles through five phases: `lesson → answering → revealed → completed → review`.

6 topic categories (rotate daily by `dayOfYear % 6`):

Slug	Title
`debits_credits`	Debits & Credits
`accounts`	Chart of Accounts
`financial_statements`	Financial Statements
`journal_entries`	Journal Entries
`bank_reconciliation`	Bank Reconciliation
`payroll`	Payroll

---

2. Critical Non-Obvious Constraints

These are the things an AI is most likely to get wrong. Read every point.

C1 — Middleware is `proxy.ts`, not `middleware.ts`

Next.js 16 renames the middleware entry point. The file at the project root is `proxy.ts`. **Do not create `middleware.ts`** — it will be silently ignored.

```
// proxy.ts — correct
export default async function proxy(request: NextRequest) { ... }
export const config = { matcher: [...] }
```

C2 — Route params and `cookies()` are async

In Next.js 16, dynamic route params are `Promise<{...}>` and `cookies()` from `next/headers` is async.

```
// CORRECT in Next.js 16
export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params
}
```

```ts
const cookieStore = await cookies()

// WRONG — will throw at runtime
const { id } = params          // no await
const cookieStore = cookies()  // no await
```

C3 — Tailwind v4 uses `@import`, not `@tailwind` directives

```ts
/* app/globals.css — CORRECT */
@import "tailwindcss";

/* WRONG — do not write these */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

There is no `tailwind.config.ts`. Tailwind v4 auto-discovers classes from `app/`, `components/`, and `lib/`.

C4 — Three Supabase clients; use the right one

File	Usage	Key used	Server or client?
`lib/supabase/client.ts`	Browser (client components, event handlers)	`anon`	Client only
`lib/supabase/server.ts`	Server components, route handlers	`anon`	Server only
`lib/supabase/admin.ts`	Admin analytics (bypasses RLS)	`service_role`	Server only — never expose to client

`createAdminClient()` must never be called from a client component or imported into the browser bundle. It only appears in `app/admin/page.tsx`.

C5 — localStorage key inconsistency (known)

Two different key formats are used — this is an existing inconsistency, not a pattern to follow:

Data	Key	Format
Session progress	`kmr_progress`	underscore
Learning Journal	`kmr-journal`	hyphen

When adding new localStorage keys, use **hyphen format** (`kmr-something`) to match the journal key. Do not change the existing keys — doing so will corrupt data for existing users.

C6 — Never change or reuse a `question.id`

`question.id` values from `data/questions/beginner.json` are stored permanently in `drill_question_answers.question_id` in Supabase. Changing or reusing an ID corrupts the analytics history for all users who answered that question.

New question IDs must follow the format: `{category_slug}_q{number}`, e.g. `debits_credits_q13`.

C7 — `optionExplanations` is parallel to `options[]`; entry at `correctIndex` should be `null`

```
{
  "options": ["A", "B", "C", "D"],
  "correctIndex": 2,
  "optionExplanations": [
    "Why A is wrong...",
    "Why B is wrong...",
    null,
    "Why D is wrong..."
  ]
}
```

The UI skips rendering a coaching note when the value is `null`. An empty string `""` is not the same as `null` — it will render an empty coaching note box.

C8 — The dual data path is intentional; do not collapse it

Anonymous users: all progress from `localStorage` via `lib/utils/progress.ts`.
Signed-in users: all progress from Supabase, fetched server-side in `app/dashboard/page.tsx`. `DashboardView.tsx` receives `user` and `dbData` props and selects the right source.

Adding a new dashboard feature means implementing it for **both paths**, not just one.

C9 — Session sync to Supabase is fire-and-forget

`DrillClient.tsx` calls `syncToDb()` after drill completion with `.catch(() => {})`. Failures are silently swallowed — this is intentional. Do not add retry logic or error UI for sync failures without a deliberate product decision.

C10 — The auth callback handles both PKCE and implicit flows

`app/auth/callback/page.tsx` checks for `?code=` (PKCE) and falls back to listening for hash tokens (implicit). Do not simplify this to one path — both flows exist in production.

C11 — `NEXT_PUBLIC_` env vars are baked into the browser bundle

`NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` are safe to use in client code. `SUPABASE_SERVICE_ROLE_KEY` and `ADMIN_EMAIL` are server-side only. Never reference a non-`NEXT_PUBLIC_` env var from a file marked `"use client"` or from any client component.

C12 — No test suite; no error monitoring

There are no automated tests (Vitest, Jest, Playwright). All testing is manual. There is no Sentry, Datadog, or equivalent. Do not write code that assumes tests will catch regressions — test manually after every non-trivial change.

---

3. Complete File Inventory

Every file in the project with its exact purpose. Files marked **[SC]** are Server Components. Files marked **[CC]** are Client Components. Files marked **[RH]** are Route Handlers.

```
keep-me-ready/
│
├── proxy.ts              [SC] Middleware — refreshes Supabase session on every request
├── next.config.ts             Standard Next.js config (minimal, no customisation)
├── postcss.config.mjs                Tailwind v4 PostCSS plugin wiring
├── eslint.config.mjs                 ESLint 9 flat config
├──  tsconfig.json                                    TypeScript  config  —
path alias "@/" maps to project root
│
├── app/
│   ├── layout.tsx          [SC] Root layout — wraps all pages with <nav> and <footer>
│   ├── page.tsx           [SC] Homepage — hero, CTA links, HomeStats, feature cards
```

```
│   ├── globals.css                Tailwind v4 import + global base styles
│   │
│   ├── admin/
│   │   └── page.tsx                    [SC]  Instructor  dashboard  —
ADMIN_EMAIL gate, fetches all
│   │                       sessions via admin client, computes 9 metrics, renders
│   │                                  InstructorDashboard
│   │
│   ├── api/
│   │   ├── scores/route.ts    [RH] POST — saves a drill session to Supabase; graceful
│   │   │                             fallback if migration 002 not yet applied
│   │   └── progress/route.ts  [RH] GET — returns aggregated stats for signed-
in user
│   │
│   ├── auth/
│   │   ├── callback/page.tsx  [CC] Handles both PKCE (?code=) and implicit (#token) flows;
│   │   │                              redirects to /dashboard on success
│   │   ├── login/page.tsx      [SC] Thin wrapper around LoginForm
│   │   └── logout/route.ts     [RH] GET — calls signOut(), redirects to /
│   │
│   ├── dashboard/
│   │   └── page.tsx              [SC] Fetches drill_sessions for signed-
in user server-side,
│   │                                  passes data to DashboardView as props
│   │
│   └── drill/
│       └── page.tsx        [SC] Thin wrapper — renders DrillClient inside a max-
width
│                                  container
│
├── components/
│   ├── DashboardView.tsx       [CC] Merges server-fetched DB data (signed-
in) with
│   │                       localStorage data (anonymous); renders all 4 dashboard
│   │                                  sections
│   ├── HomeStats.tsx       [CC] Reads localStorage on mount, shows streak/accuracy on
│   │                                                homepage;  renders  "—
" during SSR, real values on hydrate
│   ├── LoginForm.tsx       [CC] Magic link email form; Google OAuth button present but
│   │                                  inactive in v1.0
│   ├── NavUser.tsx         [CC] Subscribes to onAuthStateChange; shows email+signout or
│   │                                  sign-in link
│   ├── SaveProgressPrompt.tsx  [CC] Shown on score screen for anonymous users after a drill
│   │
│   ├── admin/
│   │   └── InstructorDashboard.tsx [SC] Pure display — renders 9 metrics from DashboardData
│   │                                      props; no client state
│   │
```

```
│   ├── dashboard/
│   │   ├── LearningJournal.tsx [CC] Reads "kmr-journal" from localStorage; renders entries
│   │   │                            newest-first with topic filter pills
│   │   ├── ProgressStats.tsx  [SC] 4-stat grid card (drills, questions, correct, accuracy)
│   │   ├── SessionHistory.tsx  [SC] Recent sessions list with accuracy bars
│   │   └── StreakBadge.tsx     [SC] Streak number + contextual message
│   │
│   └── drill/
│       ├── AnswerOption.tsx   [SC] Single answer button; green/red/dimmed on reveal
│       ├── DrillClient.tsx    [CC] Core state machine — 5 phases, manages all drill state,
│       │                            syncs to Supabase on completion
│       ├── DrillProgress.tsx  [SC] "Q n of 10" counter + live score bar (role="progressbar")
│       ├── Explanation.tsx    [SC] Coaching explanation shown after answer reveal
│       │   ├── LessonCard.tsx      [SC] Pre-drill topic overview card
│       ├── LessonReview.tsx   [CC] Post-drill per-question review with expandable coaching
│       │   ├── QuestionCard.tsx    [SC] Question text + type/category badges
│       └── ScoreDisplay.tsx   [CC] Score screen — Compare Perspectives, journal textarea,
│                                   "Save This Insight" — writes to "kmr-journal"
│
├── data/
│   └── questions/
│       ├── beginner.json        72 questions, 6 categories, 12 per category. ACTIVE.
│       ├── intermediate.json        Empty placeholder. NOT ACTIVE.
│       └── advanced.json            Empty placeholder. NOT ACTIVE.
│
├── lib/
│   ├── questions/
│   │   ├── selector.ts           selectDrillQuestions(), getQuestionById(),
│   │   │                     getQuestionsByCategory(), getTotalQuestionCount()
│   │   └── topics.ts          TOPICS record, getTodaysTopic(), getTodaysMeta(),
│   │                                ENCOURAGEMENTS, TOTAL_TOPICS (= 6)
│   │
│   ├── supabase/
│   │   ├── admin.ts             createAdminClient() — service role, server-
only
│   │   ├── client.ts            createClient() — browser, client components
│   │   └── server.ts          createClient() — server components, route handlers
│   │
│   └── utils/
│       ├── progress.ts         loadProgress(), saveSession(), getAccuracyPct()
│       │                            localStorage key: "kmr_progress"
│       └── streak.ts          computeStreakFromDates(dates: string[]): number
│
├── supabase/
│   └── migrations/
│       ├── 001_init.sql             drill_sessions table + RLS policies
│       └── 002_analytics.sql     Adds category/started_at columns, drill_question_answers
│                                    table + RLS + indexes
```

```
│
├── types/
│   └── index.ts              QuestionType, DifficultyLevel, QuestionCategory,
│                                     Question, DrillSession, UserProgress
│
└── docs/                      All project documentation (see docs/README.md)
```

---

4. TypeScript Types — Full Source

These are the exact types from `types/index.ts`. Do not invent variations.

```ts
export type QuestionType = "multiple_choice" | "true_false" | "scenario";
export type DifficultyLevel = "beginner" | "intermediate" | "advanced";

export type QuestionCategory =
  | "debits_credits"
  | "accounts"
  | "financial_statements"
  | "journal_entries"
  | "bank_reconciliation"
  | "payroll";

export interface Question {
  id: string;
  type: QuestionType;
  level: DifficultyLevel;
  category: QuestionCategory;
  text: string;
  options: string[];           // 2 for true_false, 4 for multiple_choice / scenario
  correctIndex: number;        // 0-based index into options[]
  explanation: string;
  optionExplanations?: (string | null)[];  // parallel to options[]; null at correctInd
}

export interface DrillSession {
  id: string;
  date: string;                // "YYYY-MM-DD"
  questions: string[];         // Question IDs
  answers: number[];           // Selected indices, parallel to questions[]
  score: number;
  total: number;               // default 10
  completedAt: string | null;  // ISO datetime string
}

export interface UserProgress {
  totalAnswered: number;
  totalCorrect: number;
```

```
  currentStreak: number;
  lastDrillDate: string | null;  // "YYYY-MM-DD"
  sessions: DrillSession[];      // capped at 30 in localStorage
}
```

---

5. Data Model

localStorage — anonymous users

Key	Type	Written by	Read by
`kmr_progress`	`UserProgress` (JSON)	`lib/utils/progress.ts` → `saveSession()`	`lib/utils/progress.ts` → `loadProgress()`, `DashboardView.tsx`
`kmr-journal`	`JournalEntry[]` (JSON)	`components/drill/ScoreDisplay.tsx`	`components/dashboard/LearningJournal.tsx`

JournalEntry shape (defined inline in `ScoreDisplay.tsx`, not in `types/index.ts`):

```
interface JournalEntry {
  id: string;           // `${topic.category}-${Date.now()}`
  category: string;     // QuestionCategory slug
  topicTitle: string;   // e.g. "Debits & Credits — The Foundation..."
  prompt: string;       // The reflection prompt from topics.ts
  text: string;         // Free-text insight written by the learner
  date: string;         // "YYYY-MM-DD"
  savedAt: string;      // ISO datetime string
  conceptId: string;    // Same as category slug (future-proofs mastery analytics)
}
```

Supabase — signed-in users

`drill_sessions` (created by 001, extended by 002):

Column	Type	Notes
`id`	UUID PK	auto-generated
`user_id`	UUID FK → `auth.users`	
`date`	DATE	"YYYY-MM-DD"
`score`	INTEGER	correct answers
`total`	INTEGER	questions in drill
`completed_at`	TIMESTAMPTZ	defaults to NOW()
`category`	TEXT	added by migration 002; nullable
`started_at`	TIMESTAMPTZ	added by migration 002; nullable

RLS: users can only SELECT and INSERT their own rows.

`drill_question_answers` (created by 002):

Column	Type	Notes
`id`	UUID PK	auto-generated
`session_id`	UUID FK → `drill_sessions`	CASCADE DELETE
`user_id`	UUID FK → `auth.users`	
`question_id`	TEXT	stable ID from beginner.json
`category`	TEXT	QuestionCategory slug
`was_correct`	BOOLEAN	
`answered_at`	TIMESTAMPTZ	defaults to NOW()

RLS: users can only INSERT and SELECT their own rows. Admin client bypasses both tables' RLS.

Environment variables

Variable	Visible to client?	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Yes (baked at build time)	All Supabase clients
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes (baked at build time)	All Supabase clients
`SUPABASE_SERVICE_ROLE_KEY`	No — server only	Admin client in `lib/supabase/admin.ts`
`ADMIN_EMAIL`	No — server only	Admin gate in `app/admin/page.tsx`

---

6. Component Inventory

Every component with its props interface, client/server status, and what it renders.

`DrillClient` — `components/drill/DrillClient.tsx` [CC]

No props. Fully self-contained. Calls `getTodaysMeta()` on mount to get today's topic and `selectDrillQuestions()` to get 10 questions.

Internal state:

```
type Phase = "lesson" | "answering" | "revealed" | "completed" | "review";

// key state variables:
meta: { topic: Topic; topicIndex: number; dayOfYear: number }
questions: Question[]           // 10 questions for this session
selectedAnswers: (number | null)[]  // length 10, parallel to questions
```

```
score: number
currentIndex: number
phase: Phase
drillStartedAt: string | null
```

On completion: calls `saveSession(score, total)` (localStorage) then `syncToDb()` (Supabase, fire-and-forget).

DashboardView — components/DashboardView.tsx [CC]

```tsx
interface DbDashboardData {
  totalAnswered: number;
  totalCorrect: number;
  accuracy: number | null;
  currentStreak: number;
  drillsCompleted: number;
  sessions: SessionRow[];
}

interface Props {
  user: User | null;          // from @supabase/supabase-js
  dbData: DbDashboardData | null;
}
```

If `user` is non-null, uses `dbData` (server-fetched). If `user` is null, reads localStorage on mount and constructs the same shape. Shows a pulse skeleton while localStorage loads.

InstructorDashboard — components/admin/InstructorDashboard.tsx [SC]

```tsx
interface DashboardData {
  totalUsers: number;
  activeToday: number;
  drillsCompleted: number;
  avgScore: number;
  completionRate: number;
  avgImprovement: number | null;
  avgCompletionSec: number | null;
  weeklyTrend: { label: string; avgPct: number; sessions: number }[];
  topicDifficulty: { category: string; label: string; avgPct: number; attempts: number
  missedQuestions: { questionId: string; text: string; missRate: number; attempts: numb
  hasAnswerData: boolean;
  generatedAt: string;
}
```

Pure display component. All data is computed in `app/admin/page.tsx` and passed in.

NavUser — components/NavUser.tsx [CC]

No props. Subscribes to `supabase.auth.onAuthStateChange()`. Shows email + "Sign out" when signed in, "Sign in" link when not.

LearningJournal — components/dashboard/LearningJournal.tsx [CC]

No props. Reads `localStorage["kmr-journal"]` on mount. Renders entries newest-first. Shows topic filter pills when entries span >1 category.

HomeStats — components/HomeStats.tsx [CC]

No props. Reads `localStorage["kmr_progress"]` on mount. Shows streak, accuracy %, and total questions answered. Server-renders "—" as placeholder values.

ScoreDisplay — components/drill/ScoreDisplay.tsx [CC]

```
interface Props {
  score: number;
  total: number;
  topic: Topic;
  onRestart: () => void;
  onReview: () => void;
}
```

Renders score, coaching message, Compare Perspectives section, reflection prompt, journal textarea. On "Save This Insight", writes to `localStorage["kmr-journal"]`.

LessonReview — components/drill/LessonReview.tsx [CC]

```
interface Props {
  questions: Question[];
  selectedAnswers: (number | null)[];
  topic: Topic;
  onRestart: () => void;
}
```

Renders all 10 questions with expandable per-option coaching notes and "Key Lesson Takeaways" block.

Other drill components (all SC, no complex props)

Component	Props summary
`LessonCard`	`topic: Topic, encouragement: string, onStart: () => void`
`QuestionCard`	`question: Question, index: number, total: number`
`AnswerOption`	`text: string, index: number, state: "default"|"correct"|"wrong"|"dimmed", onClick: (i: number) => void`
`Explanation`	`text: string`
`DrillProgress`	`current: number, total: number, score: number`
`ProgressStats`	`data: DbDashboardData`
`SessionHistory`	`sessions: SessionRow[]`
`StreakBadge`	`streak: number`
`LoginForm`	no props
`SaveProgressPrompt`	no props

---

7. API Routes

`POST /api/scores`

Auth required: Yes (401 if no session)

Request body:

```
{
  score: number;          // correct answers (≥0)
  total: number;          // total questions (>0)
  date: string;           // "YYYY-MM-DD"
  category?: string;      // QuestionCategory slug (requires migration 002)
  startedAt?: string;     // ISO datetime (requires migration 002)
  answers?: {
    questionId: string;
    category: string;
    wasCorrect: boolean;
  }[];
}
```

Response (201): `{ session: { id, date, score, total, completed_at } }`

Behaviour: Tries the extended insert (with `category` and `started_at`). If migration 002 is not applied, falls back to basic insert. Per-question answers are silently skipped if `drill_question_answers` does not exist.

`GET /api/progress`

Auth required: Yes (401 if no session)

Response (200):

```
{
  totalAnswered: number;
  totalCorrect: number;
  accuracy: number | null;
  currentStreak: number;
  drillsCompleted: number;
  sessions: { id: string; date: string; score: number; total: number; completed_at: str
}
```

Fetches the last 30 sessions ordered by `completed_at DESC`. Streak is computed from session dates using `computeStreakFromDates`.

`GET /auth/logout`

Auth required: No (safe to call when signed out)

Calls `supabase.auth.signOut()` and redirects to `/`.

---

8. Exact Code Patterns to Follow

Copy these patterns exactly when adding new functionality. Deviate only when a constraint requires it.

Pattern A — Create a server-side Supabase client

```ts
// In a Server Component or Route Handler
import { createClient } from "@/lib/supabase/server";

const supabase = await createClient();
const { data: { user } } = await supabase.auth.getUser();
```

Pattern B — Create a client-side Supabase client (in an effect or handler, never at module scope)

```ts
// In a Client Component — inside useEffect or an event handler only
import { createClient } from "@/lib/supabase/client";

useEffect(() => {
  const supabase = createClient();
```

```
  // ...
}, []);
```

Pattern C — Protect a server component with auth + admin check

```
import { redirect } from "next/navigation";
import { createClient } from "@/lib/supabase/server";

export default async function ProtectedPage() {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) redirect("/auth/login");

  const adminEmail = process.env.ADMIN_EMAIL;
  if (!adminEmail || user.email !== adminEmail) {
    return <div>Access Denied</div>;
  }
  // ...
}
```

Pattern D — Read from localStorage safely (guards for SSR)

```
// lib/utils/progress.ts pattern — always guard typeof window
export function loadProgress(): UserProgress {
  if (typeof window === "undefined") return { ...DEFAULT_PROGRESS };
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return { ...DEFAULT_PROGRESS };
    return JSON.parse(raw) as UserProgress;
  } catch {
    return { ...DEFAULT_PROGRESS };
  }
}
```

Pattern E — Add a new question to the question bank

```
{
  "id": "debits_credits_q13",
  "type": "multiple_choice",
  "level": "beginner",
  "category": "debits_credits",
  "text": "Your question text here.",
  "options": ["Option A", "Option B", "Option C", "Option D"],
```

```
  "correctIndex": 2,
  "explanation": "Coaching explanation shown after the answer. Written in mentor tone,
  "optionExplanations": [
    "Why A is the wrong choice and what misconception it reflects.",
    "Why B is the wrong choice and what misconception it reflects.",
    null,
    "Why D is the wrong choice and what misconception it reflects."
  ]
}
```

Rules: - `id` must be globally unique across the entire question bank. Never reuse. - `correctIndex` is 0-based. Validate it is within `options.length - 1`. - `optionExplanations[correctIndex]` must be `null`. - `explanation` must be non-empty and coaching-toned (see Section 10). - All `optionExplanations` except at `correctIndex` must be non-empty strings.

Pattern F — Add a new database column (migration file)

```
-- supabase/migrations/003_your_feature.sql
-- Run this in the Supabase SQL Editor after 002_analytics.sql

ALTER TABLE public.drill_sessions ADD COLUMN IF NOT EXISTS new_column TEXT;
```

Rules: - Always use `IF NOT EXISTS` / `IF NOT EXISTS` guards. - Never modify existing migration files — they are the historical record. - Number sequentially: `003_`, `004_`, etc. - Apply to the dev Supabase project first; apply to production after the code change deploys.

Pattern G — Compute today's topic (the only correct way)

```
import { getTodaysTopic, getTodaysMeta } from "@/lib/questions/topics";

// Just the topic:
const topic = getTodaysTopic();

// Topic + index + dayOfYear (needed for DrillClient):
const { topic, topicIndex, dayOfYear } = getTodaysMeta();
```

Do not recompute the day rotation formula inline. All callers must go through `lib/questions/topics.ts`.

Pattern H — Streak computation (the only correct way)

```
import { computeStreakFromDates } from "@/lib/utils/streak";
```

```
const streak = computeStreakFromDates(sessions.map(s => s.date));
// dates is string[] of "YYYY-MM-DD" values (duplicates are handled internally)
```

Do not write streak logic inline. The function handles: no sessions → 0; most recent session older than yesterday → 0; consecutive days → increments; same-day duplicates → deduped automatically.

---

9. Product Rules

These rules reflect deliberate product decisions. Do not override them without an explicit instruction to do so.

What to build

- Features that serve one of the five personas: Sarah (job seeker), Marcus (freelancer), Priya (career changer), David (small business owner), Instructor
- Features that are consistent with the coaching-not-assessment philosophy
- Features that make the daily habit more visible or sustainable (streak calendar, journal milestones, etc.)
- Features that deepen the existing experience (better explanations, more question types, mastery tracking)

What NOT to build without explicit instruction

Feature	Reason
Countdown timers during questions	Creates anxiety incompatible with coaching mindset
Point deductions for wrong answers	Penalises the learning process
Competitive leaderboards or rankings	Undermines personal practice framing
“Your streak is about to break!” notifications	Anxiety-driven manipulation
Paywalled explanations or coaching notes	Betrays the core educational value
AI-generated question content	All questions are human-authored for quality
Social profiles or public scores	Not part of the personal practice model

Anonymous users must have the full experience

Anonymous users get 100% of the drill and feedback experience. Never degrade the anonymous experience to force sign-up. The “Save your progress” prompt is an offer, not a gate.

---

10. Copy and Tone Rules

Every user-facing string must follow these rules. They are not style preferences — they are product requirements.

Coaching voice, not assessment voice

The product speaks like a skilled mentor who wants you to understand, not a teacher marking your paper.

Wrong (assessment): > "Incorrect. The correct answer is C."

Right (coaching): > "Almost — the key here is that debits always increase asset accounts. Once that clicks, the rest of the debit/credit rules fall into place."

Explanations address the concept, not the answer choice

Coaching notes explain *why* the concept works, not just *why option X was wrong*.

Wrong: > "A is wrong because debit does not mean decrease."

Right: > "Debit and credit don't carry positive or negative meaning in bookkeeping — they only describe which side of an account is affected. An asset account increases with a debit, which is actually a good thing for cash."

Score screen messages acknowledge the work without shame

Do not use language that implies failure. Even a low score is a coaching opportunity.

Wrong: > "You failed this drill. Try harder next time."

Right (for a low score): > "This topic is still building in your mind — that's exactly what drills are for. Come back tomorrow and watch it get clearer."

Journal confirmations emphasise growth, not completion

Wrong: > "Saved!"

Right: > "Added to your Learning Journal. Every reflection you save becomes part of your professional growth."

Second person, present tense

All explanations and coaching notes are written to "you" in the present tense.

Wrong: > "Students often confuse Accounts Receivable with Accounts Payable."

Right: > “These two accounts are easy to mix up — Accounts Receivable is money owed to *you,* while Accounts Payable is money *you* owe others.”

Keep Me Ready — Practice makes permanent.